

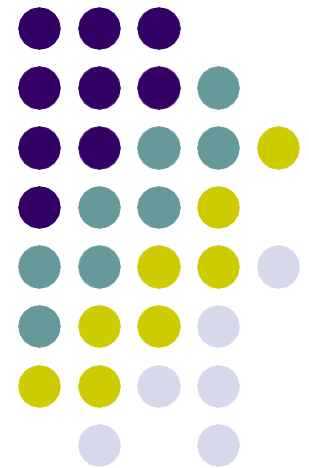
Digital Image Processing (CS/ECE 545)

Lecture 4: Filters (Part 2)

& Edges and Contours

Prof Emmanuel Agu

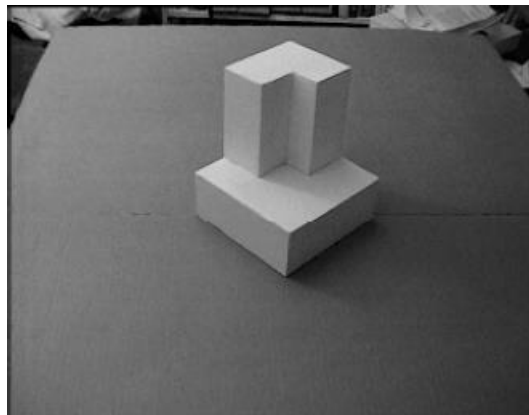
*Computer Science Dept.
Worcester Polytechnic Institute (WPI)*





What is an Edge?

- Edge? sharp change in brightness (discontinuities)
- Where do edges occur?
 - **Actual edges:** Boundaries between objects
 - Sharp change in brightness can also occur within object
 - Reflectance changes
 - Change in surface orientation
 - Illumination changes. E.g. Cast shadow boundary



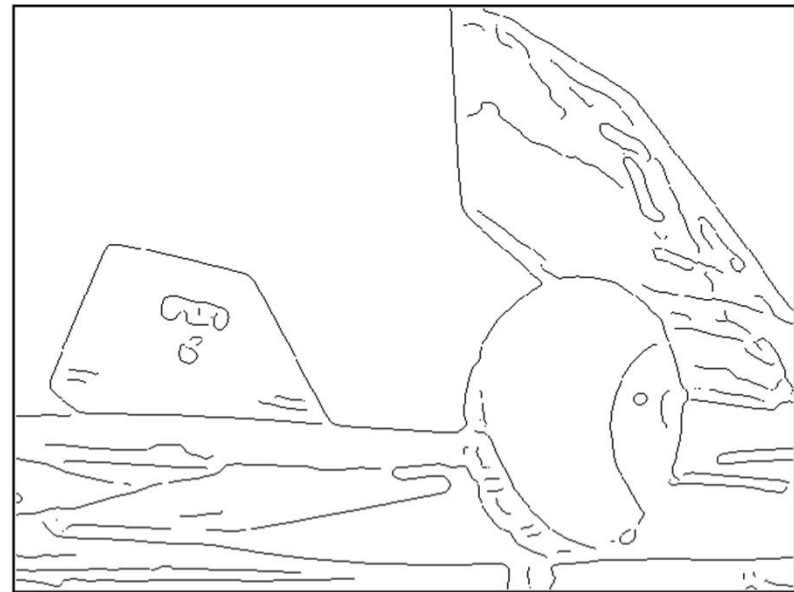


Edge Detection

- Image processing task that finds edges and contours in images
- Edges so important that human vision can reconstruct edge lines



(a)

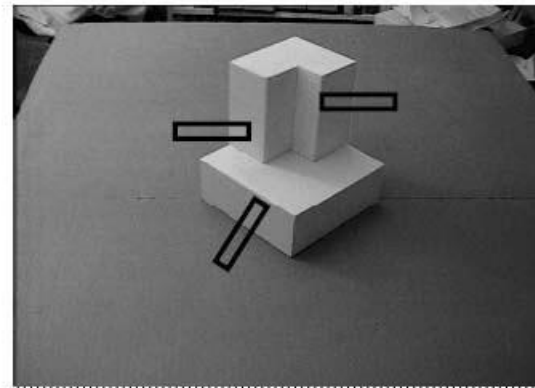
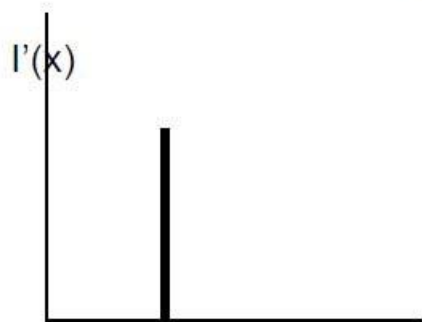
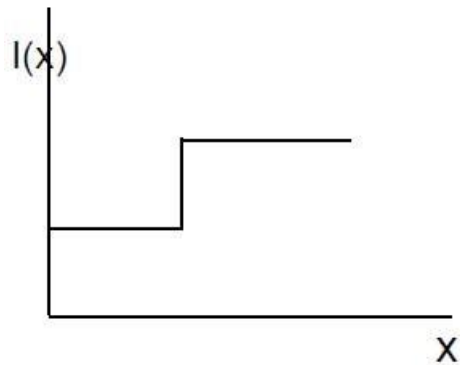


(b)



Characteristics of an Edge

- Edge: A sharp change in brightness
- Ideal edge is a step function in some direction

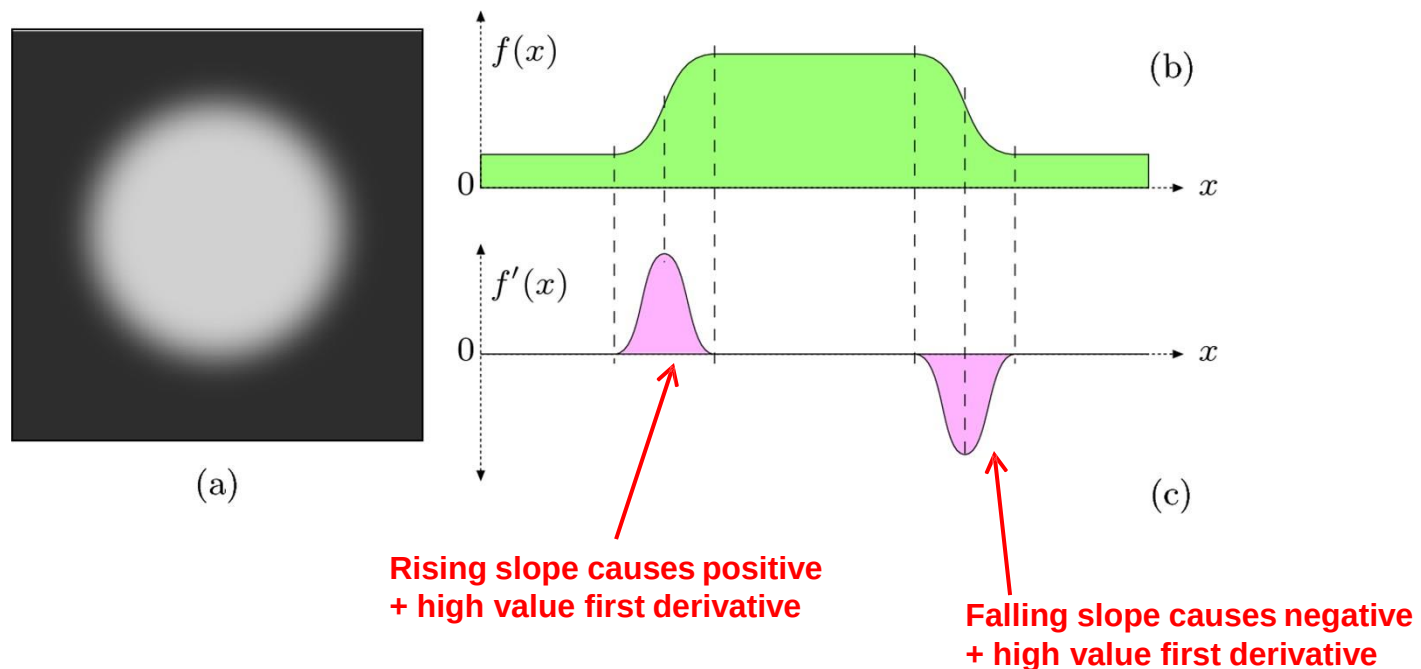




Characteristics of an Edge

- Real (non-ideal) edge is a slightly blurred step function
- Edges can be characterized by high value first derivative

$$f'(x) = \frac{df}{dx}(x)$$

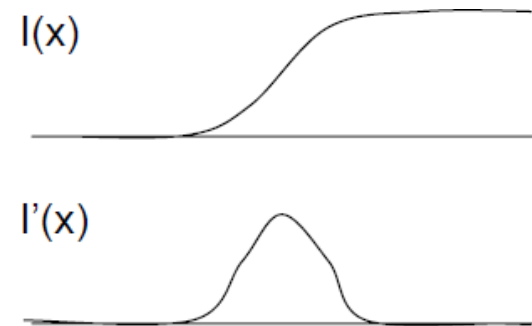
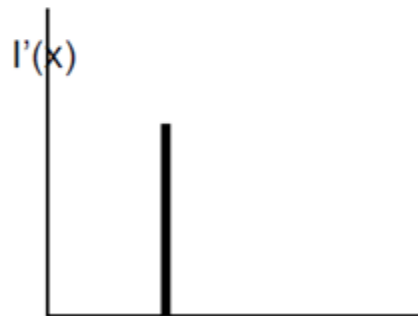
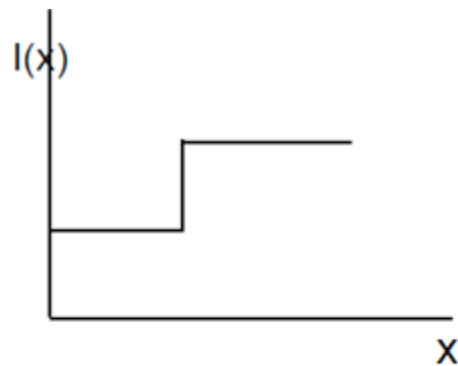




Characteristics of an Edge

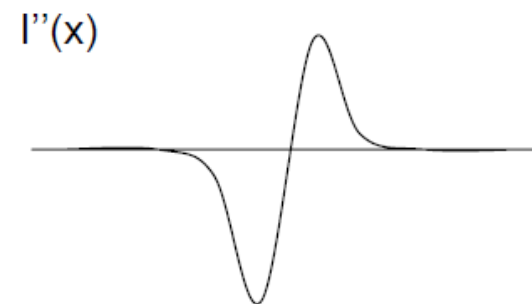
- Ideal edge is a step function in certain direction.
- First derivative of $I(x)$ has a **peak** at the edge
- Second derivative of $I(x)$ has a **zero crossing** at edge

Ideal edge



Real edge

First derivative shows peak

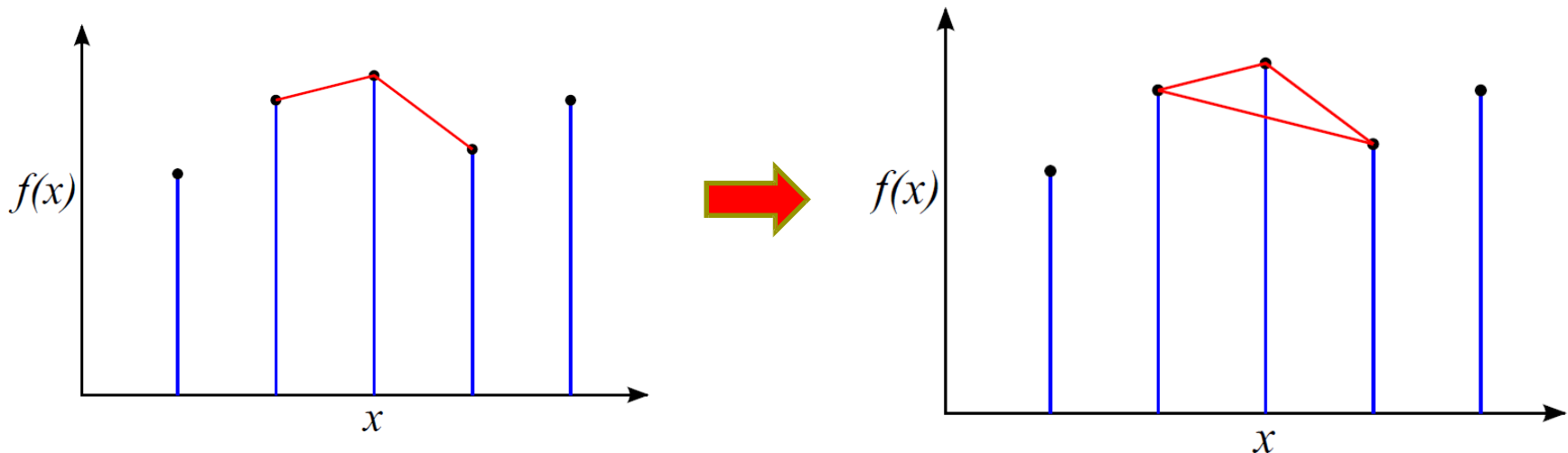


Second derivative shows zero crossing



Slopes of Discrete Functions

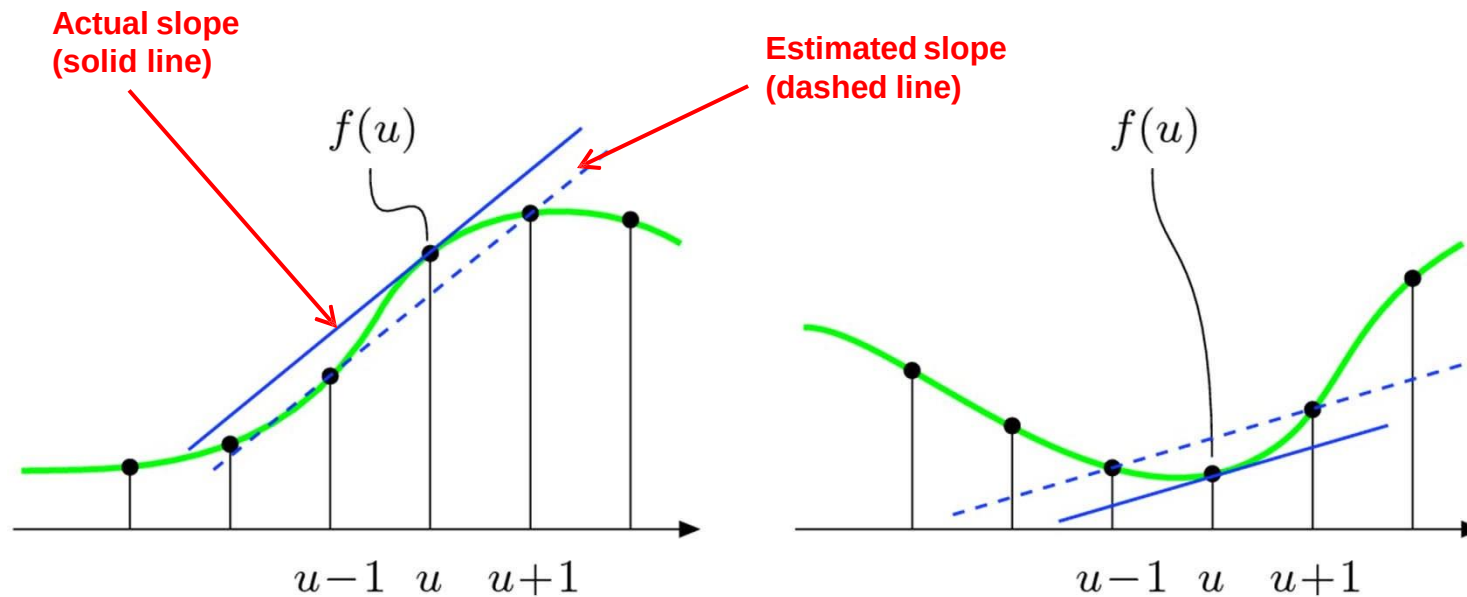
- Left and right slope may not be same
- Solution? Take average of left and right slope





Computing Derivative of Discrete Function

$$\frac{df}{du}(u) \approx \frac{f(u+1) - f(u-1)}{2} = 0.5 \cdot (f(u+1) - f(u-1))$$





Finite Differences

- Forward difference (right slope)

$$\Delta_+ f(x) = f(x + 1) - f(x)$$

- Backward difference (left slope)

$$\Delta_- f(x) = f(x) - f(x - 1)$$

- Central Difference (average slope)

$$\Delta f(x) = \frac{1}{2} (f(x + 1) - f(x - 1))$$



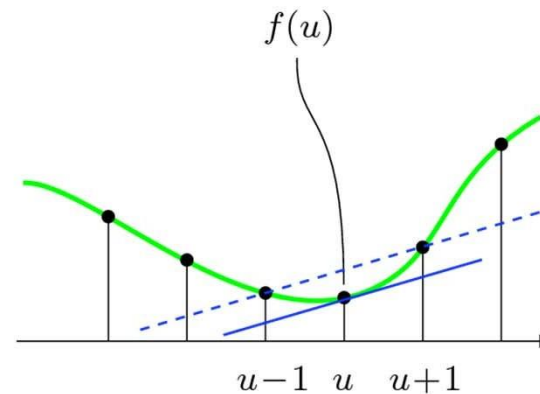
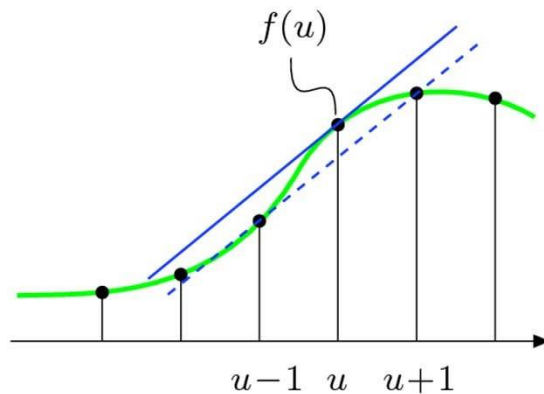
Derivative Filters

- Recall that we can compute derivative of discrete function as

$$\frac{df}{du}(u) \approx \frac{f(u+1) - f(u-1)}{2} = 0.5 \cdot (f(u+1) - f(u-1))$$

- Can we make linear filter that computes central differences

$$H_x^D = [-0.5 \quad 0 \quad 0.5] = 0.5 \cdot [-1 \quad 0 \quad 1]$$





Finite Differences as Convolutions

- Forward difference

$$\Delta_+ f(x) = f(x + 1) - f(x)$$

- Take a convolution kernel $H = [0 \quad -1 \quad 1]$

$$\Delta_+ f = f * H$$



Finite Differences as Convolutions

- Central difference

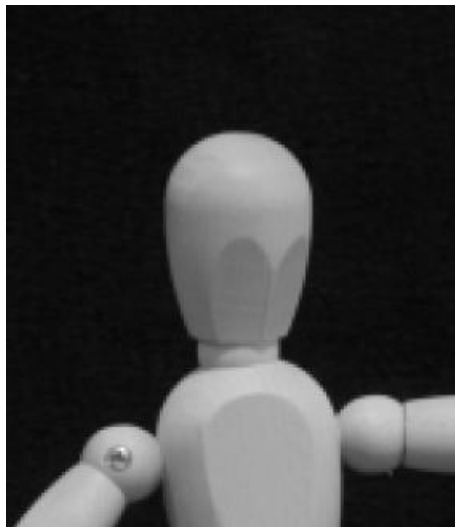
$$\Delta f(x) = \frac{1}{2} (f(x+1) - f(x-1))$$

- Convolution kernel is: $H = [-0.5 \quad 0 \quad 0.5]$

$$\Delta f(x) = f * H$$

- **Notice:** Derivative kernels sum to zero

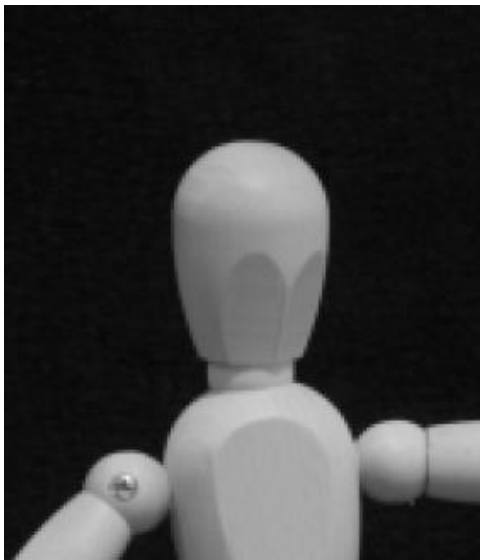
x-Derivative of Image using Central Difference



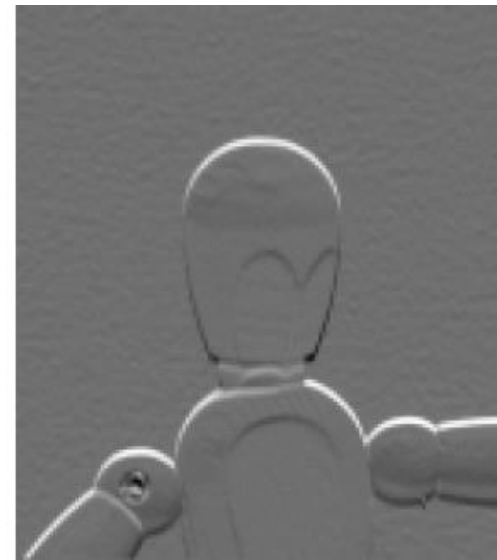
$$* \begin{bmatrix} -0.5 & 0 & 0.5 \end{bmatrix} =$$



y-Derivative of Image using Central Difference



$$* \begin{bmatrix} -0.5 \\ 0 \\ 0.5 \end{bmatrix} =$$



```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from scipy.ndimage import convolve1d

# Load the image in grayscale
image = cv2.imread('image2.png', cv2.IMREAD_GRAYSCALE)

# Define the 1D derivative kernel
kernel = np.array([-0.5, 0, 0.5])

# Apply convolution in X and Y directions
gradient_x = convolve1d(image.astype(float), kernel, axis=1)
gradient_y = convolve1d(image.astype(float), kernel, axis=0)

# Compute gradient magnitude
gradient_mag = np.sqrt(gradient_x**2 + gradient_y**2)

# Convert all to 8-bit for display
gradient_x_disp = cv2.convertScaleAbs(gradient_x)
gradient_y_disp = cv2.convertScaleAbs(gradient_y)
gradient_mag_disp = cv2.convertScaleAbs(gradient_mag)
```



```
# Display all results
titles = ['Original', 'Gradient X', 'Gradient Y', 'Gradient Magnitude']
images = [image, gradient_x_disp, gradient_y_disp, gradient_mag_disp]
```

```
plt.figure(figsize=(14, 6))
for i in range(4):
    plt.subplot(1, 4, i + 1)
    plt.imshow(images[i], cmap='gray')
    plt.title(titles[i])
    plt.axis('off')
```

```
plt.tight_layout()
plt.show()
```

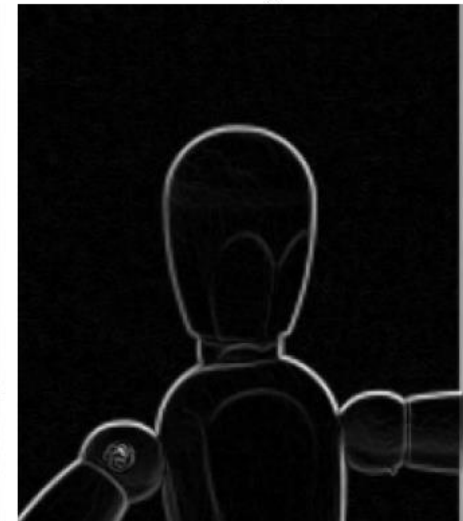
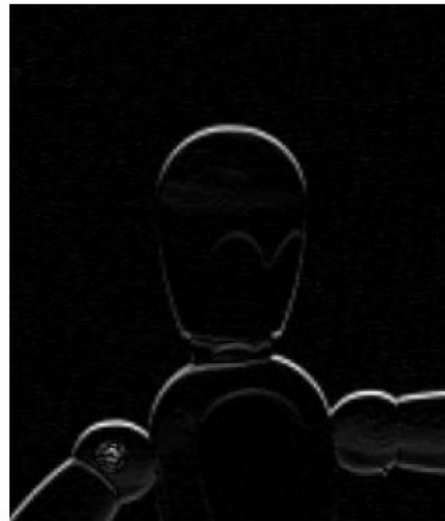
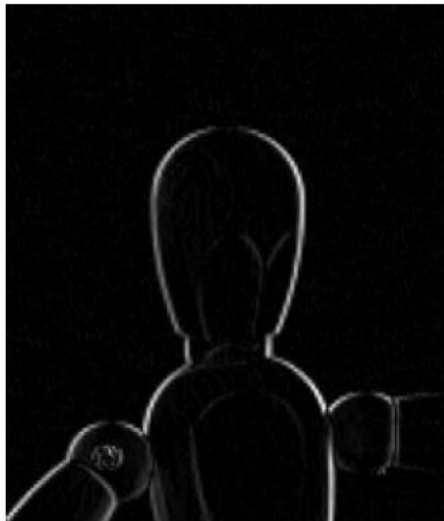
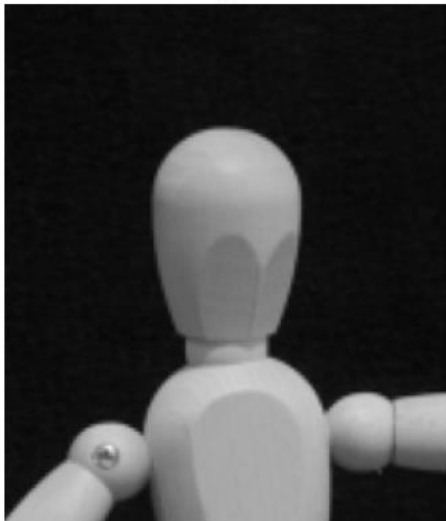


Original

Gradient X

Gradient Y

Gradient Magnitude



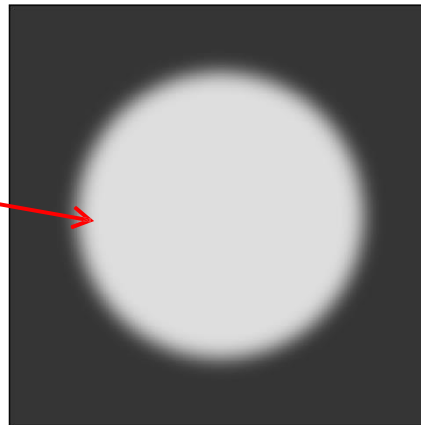
Derivative Filters



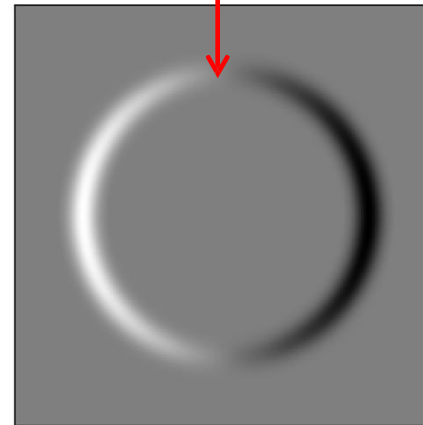
$$H_x^D = \begin{bmatrix} -0.5 & 0 & 0.5 \end{bmatrix} = 0.5 \cdot \begin{bmatrix} -1 & 0 & 1 \end{bmatrix}$$

Gradient slope in
horizontal direction

A synthetic
image



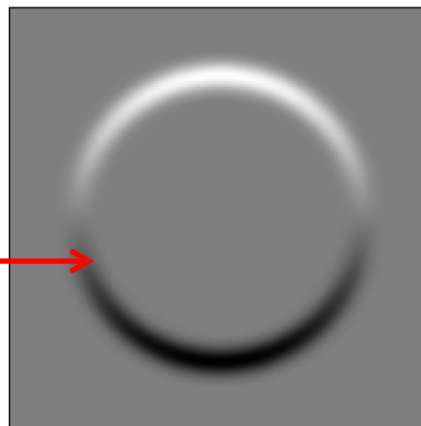
(a)



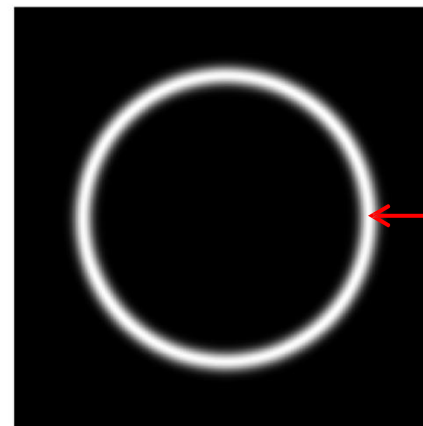
(b)

$$H_y^D = \begin{bmatrix} -0.5 \\ 0 \\ 0.5 \end{bmatrix} = 0.5 \cdot \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix}$$

Gradient slope in
vertical direction



(c)



(d)

Magnitude of
gradient



Partial Image Derivatives

- Partial derivatives of images replaced by finite differences

$$\Delta_x f = f(x, y) - f(x - 1, y) \quad \begin{bmatrix} -1 & 1 \end{bmatrix} \quad \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

$$\Delta_y f = f(x, y) - f(x, y - 1)$$

- Alternatives are:

$$\Delta_{2x} f = f(x + 1, y) - f(x - 1, y) \quad \begin{bmatrix} -1 & 0 & 1 \end{bmatrix} \quad \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix}$$

$$\Delta_{2y} f = f(x, y + 1) - f(x, y - 1)$$

- Robert's gradient

$$\Delta_+ f = f(x + 1, y + 1) - f(x, y) \quad \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$$

$$\Delta_- f = f(x, y + 1) - f(x + 1, y) \quad \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

$$\begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

Prewitt

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Sobel



Prewitt and Sobel Edge Operators

- Prewitt Operator

$$H_x^P = \begin{bmatrix} -1 & 0 & 1 \\ -1 & \mathbf{0} & 1 \\ -1 & 0 & 1 \end{bmatrix} \quad \text{and} \quad H_y^P = \begin{bmatrix} -1 & -1 & -1 \\ 0 & \mathbf{0} & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

Written in separable form $\rightarrow$ $H_x^P = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} * \begin{bmatrix} -1 & 0 & 1 \end{bmatrix}$ and $H_y^P = \begin{bmatrix} 1 & 1 & 1 \end{bmatrix} * \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix}$

- Sobel Operator

$$H_x^S = \begin{bmatrix} -1 & 0 & 1 \\ -2 & \mathbf{0} & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad \text{and} \quad H_y^S = \begin{bmatrix} -1 & -2 & -1 \\ 0 & \mathbf{0} & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

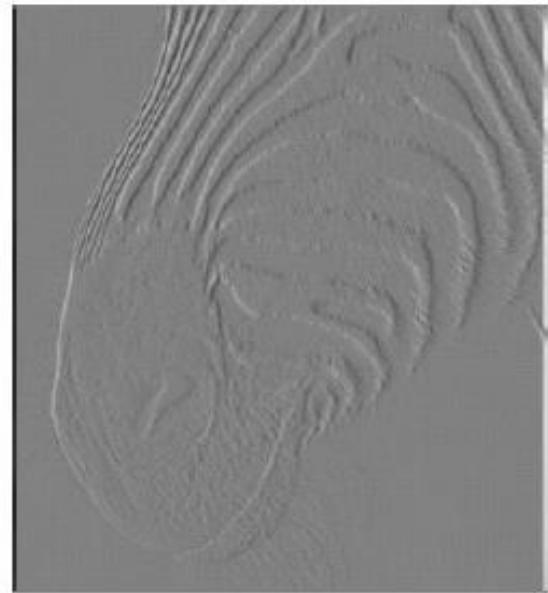
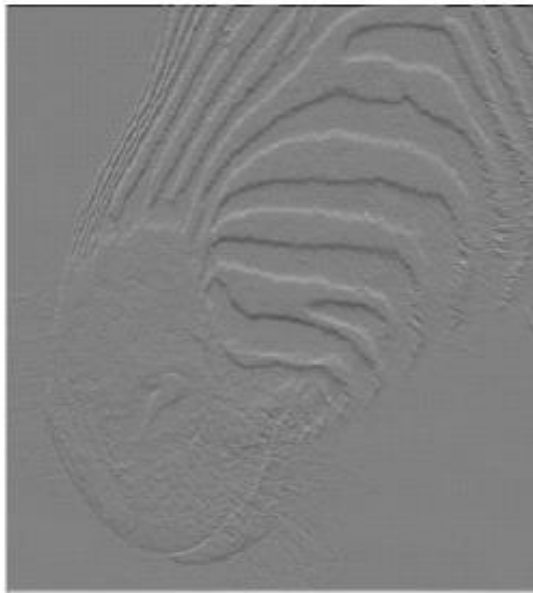


Improved Sobel Filter

- Original Sobel filter relatively inaccurate
- Improved versions proposed by Jahne

$$H_x^{S'} = \frac{1}{32} \begin{bmatrix} -3 & 0 & 3 \\ -10 & \mathbf{0} & 10 \\ -3 & 0 & 3 \end{bmatrix} \quad \text{and} \quad H_y^{S'} = \frac{1}{32} \begin{bmatrix} -3 & -10 & -3 \\ 0 & \mathbf{0} & 0 \\ 3 & 10 & 3 \end{bmatrix}$$

Prewitt and Sobel Edge Operators





Compass Operators

- Linear edge filters involve trade-off

Sensitivity to
Edge magnitude ↑ = ↓ Sensitivity to
orientation

- Example: Prewitt and Sobel operators detect edge magnitudes but use only 2 directions (insensitive to orientation)
- Solution? Use many filters, each sensitive to narrow range of orientations (**compass operators**)



Compass Operators

- Edge operators proposed by Kirsh uses 8 filters with orientations spaced at 45 degrees

$$H_0^K = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

$$H_4^K = \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}$$

$$H_1^K = \begin{bmatrix} -2 & -1 & 0 \\ -1 & 0 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

$$H_5^K = \begin{bmatrix} 2 & 1 & 0 \\ 1 & 0 & -1 \\ 0 & -1 & -2 \end{bmatrix}$$

$$H_2^K = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

$$H_6^K = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

$$H_3^K = \begin{bmatrix} 0 & -1 & -2 \\ 1 & 0 & -1 \\ 2 & 1 & 0 \end{bmatrix}$$

$$H_7^K = \begin{bmatrix} 0 & 1 & 2 \\ -1 & 0 & 1 \\ -2 & -1 & 0 \end{bmatrix}$$

Need only to compute 4 filters
Since $H_4 = -H_0$, etc



Laplacian Operator

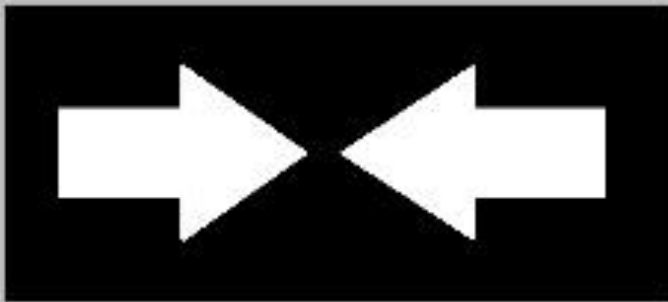
- The major difference between Laplacian and other operators is that unlike other operators Laplacian didn't take out edges in any particular direction but it take out edges in following classification:
 - ✓ Inward Edges
 - ✓ Outward Edges

$$H = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

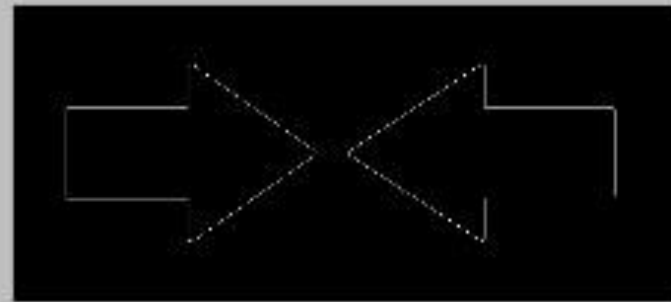
Compass Operators



Original



Laplacian



Sobel X



Sobel Y



```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load the image in grayscale
image = cv2.imread('image1.png', cv2.IMREAD_GRAYSCALE)

# Apply Laplacian operator
laplacian = cv2.Laplacian(image, cv2.CV_64F)

# Apply Sobel operator in X direction
sobel_x = cv2.Sobel(image, cv2.CV_64F, dx=1, dy=0, ksize=3)

# Apply Sobel operator in Y direction
sobel_y = cv2.Sobel(image, cv2.CV_64F, dx=0, dy=1, ksize=3)

# Convert float64 images to uint8 for display
laplacian = cv2.convertScaleAbs(laplacian)
sobel_x = cv2.convertScaleAbs(sobel_x)
sobel_y = cv2.convertScaleAbs(sobel_y)
```



```
# Display results using matplotlib
titles = ['Original', 'Laplacian', 'Sobel X', 'Sobel Y']
images = [image, laplacian, sobel_x, sobel_y]
```

```
plt.figure(figsize=(12, 6))
for i in range(4):
    plt.subplot(1, 4, i + 1)
    plt.imshow(images[i], cmap='gray')
    plt.title(titles[i])
    plt.axis('off')
```



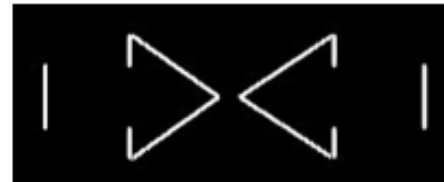
Original



Laplacian



Sobel X



Sobel Y

